

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

Dr VIMAL V R

QUESTIONS: 5

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

Annexure A

PSEUDOCODE WRITING TASK

S.No.	Questions	Mark
1	A system is at risk of disk overflow due to continuously increasing log files. Analyze the situation and develop a file system space management algorithm, using pseudocode, to monitor disk usage and automatically delete or rotate log files whenever disk utilization exceeds 80%. Pseudocode:BEGIN SET threshold = 80% usage = GET_DISK_USAGE() IF usage > threshold THEN IDENTIFY log_files SORTED BY size DESC FOR each file IN log_files DO DELETE or COMPRESS file usage = GET_DISK_USAGE() IF usage <= threshold THEN BREAK ENDIF ENDFOR ENDIF END	5
2	Improve CPU efficiency in a system by designing a priority-based scheduling algorithm. Develop and present the corresponding pseudocode to demonstrate how processes are selected and executed based on their priority levels. Pseudocode:BEGIN INPUT process_list SORT process_list BY priority (highest first) WHILE process_list NOT EMPTY DO SELECT process WITH highest priority ALLOCATE CPU time EXECUTE process IF process NOT completed THEN REDUCE priority slightly ADD back to process_list ENDIF ENDWHILE END	5
3	Efficiently manage and distribute RAM resources among multiple virtual machines in a system to ensure balanced performance and optimal utilization. Pseudocode:BEGIN total_RAM = 16GB host_reserved = 4GB available_RAM = total_RAM - host_reserved INPUT number_of_VMs min_RAM = 2GB FOR i = 1 TO number_of_VMs DO	5

	<pre> IF available_RAM >= min_RAM THEN ALLOCATE min_RAM to VM[i] available_RAM = available_RAM - min_RAM ELSE PRINT "Insufficient memory" ENDIF </pre>	
4	Select the most appropriate hypervisor for a given system environment by analyzing the specified requirements and constraints, and justify your decision to ensure optimal performance and efficient resource utilization. Pseudocode: BEGIN <pre> INPUT performance_required, budget, hardware_access IF performance_required = HIGH AND hardware_access = TRUE THEN SELECT Type_1_Hypervisor ELSE IF budget = LOW THEN SELECT Type_2_Hypervisor ELSE SELECT Type_1_Hypervisor ENDIF PRINT selected_hypervisor END </pre>	5
5	Manage and regulate CPU and memory usage of applications in a system by analyzing the scenario and implementing suitable resource-limiting mechanisms to ensure balanced and efficient system performance. Pseudocode: BEGIN <pre> INPUT application_list FOR each app IN application_list DO CREATE cgroup for app SET CPU limit SET memory limit ASSIGN process to cgroup ENDFOR END </pre>	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent	Mostly clear with minor	Difficult to follow in parts	Poorly structured and

		format, easy to follow	issues		unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1 – File System Space Management

Problem Understanding

A system may face disk overflow when log files continuously increase in size. The algorithm should continuously monitor disk utilization. When usage exceeds **80%**, the system should identify large log files and compress, rotate, or delete old files until disk utilization falls below the threshold.

Pseudocode

BEGIN

SET threshold = 80%

WHILE system is running DO

 usage = GET_DISK_USAGE()

 IF usage > threshold THEN

 PRINT "Warning: Disk usage exceeds 80%"

 log_files = GET_LOG_FILES()

 SORT log_files BY size DESCENDING

 FOR each file IN log_files DO

 IF file is old THEN

 ROTATE file

 ELSE IF file can be compressed THEN

 COMPRESS file

 ELSE

 DELETE oldest file

 ENDIF

 usage = GET_DISK_USAGE()

```

    IF usage <= threshold THEN

        BREAK

    ENDIF

ENDFOR

IF usage > threshold THEN

    PRINT "Warning: Disk space is still critically low"

ELSE

    PRINT "Disk usage reduced successfully"

ENDIF

ENDIF

WAIT for monitoring interval

ENDWHILE

END

```

Input: Disk usage, log files, file size, file age

Output: Reduced disk utilization and managed log files

Constraint: Disk utilization should be maintained at or below 80%.

Question 2 – Priority-Based CPU Scheduling

Problem Understanding

Processes have different priority levels. The CPU should execute the process with the highest priority first. If a process is not completed, its priority can be slightly reduced to prevent lower-priority processes from waiting indefinitely.

Pseudocode

```

BEGIN

INPUT process_list

FOR each process IN process_list DO

```

```

INPUT process.priority

INPUT process.burst_time

SET process.remaining_time = process.burst_time

ENDFOR

WHILE process_list is NOT EMPTY DO

    SORT process_list BY priority DESCENDING

    SELECT process WITH highest priority

    ALLOCATE CPU to selected process

    EXECUTE selected process

    IF selected process is completed THEN

        REMOVE selected process FROM process_list

        PRINT "Process completed"

    ELSE

        REDUCE selected process priority slightly

        UPDATE selected process remaining_time

        RETURN selected process TO process_list

    ENDIF

ENDWHILE

PRINT "All processes completed"

END

```

Input: Process ID, priority, burst time

Output: Order of process execution and completion

Objective: Improve CPU utilization while ensuring processes are executed according to priority.

Question 3 – RAM Resource Management for Virtual Machines

Problem Understanding

A host system has limited RAM that must be distributed among multiple virtual machines (VMs). Some memory must be reserved for the host operating system. Each VM receives a minimum amount of RAM, and remaining memory can be distributed fairly.

Pseudocode

BEGIN

SET total_RAM = 16 GB

SET host_reserved = 4 GB

SET available_RAM = total_RAM - host_reserved

INPUT number_of_VMs

SET min_RAM = 2 GB

IF number_of_VMs * min_RAM > available_RAM THEN

 PRINT "Insufficient RAM for all virtual machines"

ELSE

 FOR i = 1 TO number_of_VMs DO

 ALLOCATE min_RAM TO VM[i]

 available_RAM = available_RAM - min_RAM

 ENDFOR

 IF available_RAM > 0 THEN

 extra_RAM = available_RAM / number_of_VMs

 FOR i = 1 TO number_of_VMs DO

 ALLOCATE extra_RAM TO VM[i]


```

    ENDFOR

    SET available_RAM = 0

ENDIF

PRINT "RAM allocated successfully"

FOR i = 1 TO number_of_VMs DO

    PRINT "VM", i, "RAM allocation"

ENDFOR

ENDIF

END

```

Input: Total RAM, host-reserved RAM, number of VMs

Output: RAM allocation for each VM

Constraint: Host RAM must remain reserved and every VM should receive at least 2 GB.

Example:

Total RAM = 16 GB

Host reserved = 4 GB

Available = 12 GB

For 4 VMs, each VM initially receives **2 GB**, with the remaining **4 GB** distributed equally. Therefore, each VM receives **3 GB**.

Question 4 – Hypervisor Selection

Problem Understanding

A hypervisor allows multiple virtual machines to run on the same physical system. The selection depends on performance requirements, budget, and whether direct hardware access is required.

- **Type 1 Hypervisor:** Runs directly on physical hardware and provides better performance.

- **Type 2 Hypervisor:** Runs on top of a host operating system and is generally suitable for development, testing, and lower-cost environments.

Pseudocode

BEGIN

INPUT performance_required

INPUT budget

INPUT hardware_access

INPUT environment

IF performance_required = HIGH AND hardware_access = TRUE THEN

 selected_hypervisor = "Type 1 Hypervisor"

ELSE IF environment = "Production" THEN

 selected_hypervisor = "Type 1 Hypervisor"

ELSE IF budget = LOW AND performance_required = LOW THEN

 selected_hypervisor = "Type 2 Hypervisor"

ELSE

 selected_hypervisor = "Type 2 Hypervisor"

ENDIF

PRINT "Selected Hypervisor: ", selected_hypervisor

IF selected_hypervisor = "Type 1 Hypervisor" THEN

 PRINT "Suitable for high performance and production environments"

ELSE

 PRINT "Suitable for development, testing and low-cost environments"

ENDIF

END

Input: Performance requirement, budget, hardware access, environment

Output: Appropriate hypervisor type

Decision: Type 1 is preferred for high-performance and production environments, while Type 2 is suitable for development/testing and lower-cost requirements.

Question 5 – CPU and Memory Resource Limiting

Problem Understanding

Multiple applications may consume excessive CPU and memory, affecting overall system performance. Linux **control groups (cgroups)** can be used to limit and regulate the resources available to each application.

Pseudocode

BEGIN

INPUT application_list

FOR each app IN application_list DO

 CREATE cgroup FOR app

 INPUT CPU_limit

 INPUT memory_limit

 SET CPU limit FOR app

 SET memory limit FOR app

 ASSIGN app processes TO cgroup

 MONITOR CPU usage

 MONITOR memory usage

 IF CPU usage > CPU_limit THEN

 THROTTLE CPU usage

 ENDIF

 IF memory usage > memory_limit THEN

LIMIT memory allocation

ENDIF

IF application exceeds limits continuously THE

LOG resource violation

ENDIF

ENDFOR

PRINT "Resource limits applied successfully"

END

Input: Application list, CPU limit, memory limit

Output: Controlled CPU and memory usage

Linux Mechanism Used: cgroups

Objective: Prevent one application from consuming excessive system resources and maintain balanced system performance.